3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

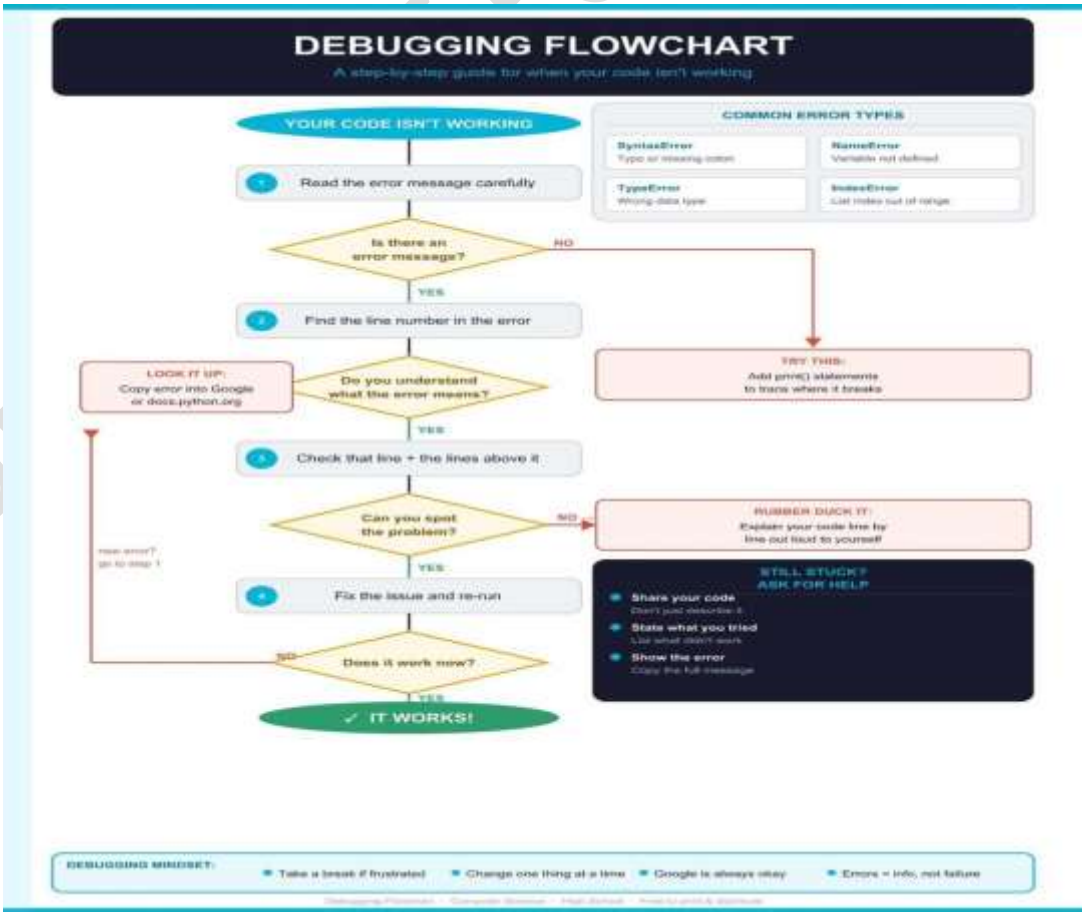
Classes : Offline & Online

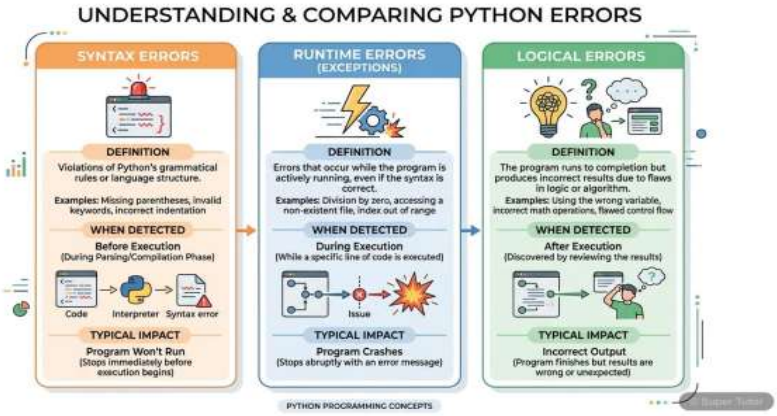
Daily Teaching : 2 Hours

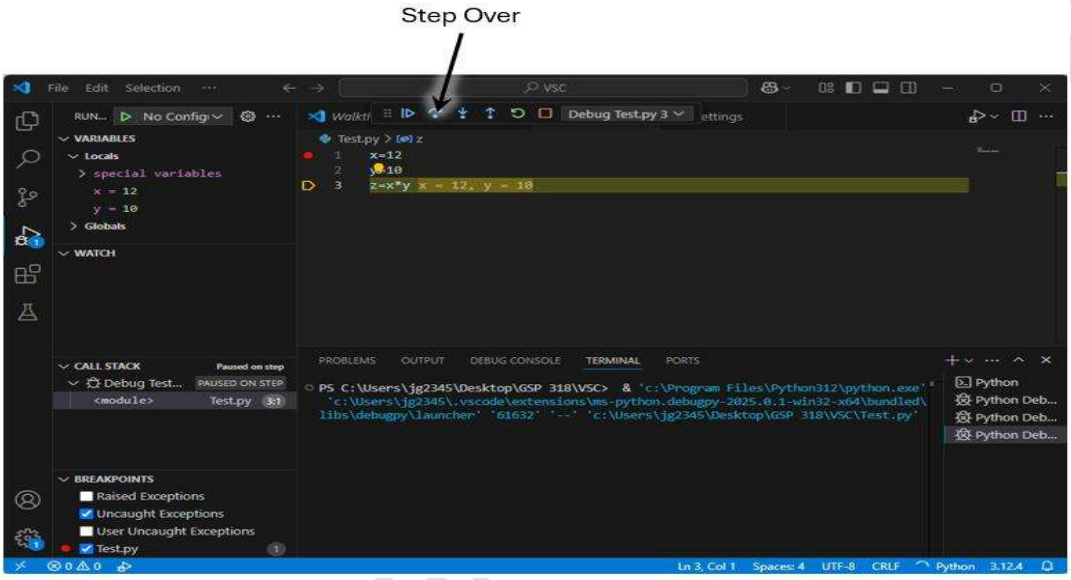
Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 18 – Debugging Python Programs







1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand what debugging means.
- Identify different types of programming errors.
- Read Python error messages and tracebacks.
- Find syntax, runtime and logical errors.
- Use print() for debugging.
- Use breakpoints and the Python debugger.
- Understand common Python errors.

- Debug functions, loops, lists, dictionaries and files.
- Apply a systematic debugging process.
- Fix errors without changing the intended logic of a program.

2. What is Debugging?

Debugging is the process of **finding, understanding and fixing errors in a program**.

A mistake in a program is commonly called a **bug**.

Program

↓

Run Program

↓

Error / Wrong Output

↓

Find the Problem

↓

Understand the Cause

↓

Fix the Code

↓

Test Again

Simple definition

Debugging = Find the bug + Understand it + Fix it + Test again

3DWEBSOFT FOUNDATION

3. What is a Bug?

A **bug** is a mistake or defect in a program that causes:

- An error
- Unexpected output
- Incorrect calculations
- Program crashes
- Incorrect behavior

Example:

a = 10

b = 20

print(a - b)

If the programmer wanted addition, the program runs successfully but produces the wrong result.

Expected:

30

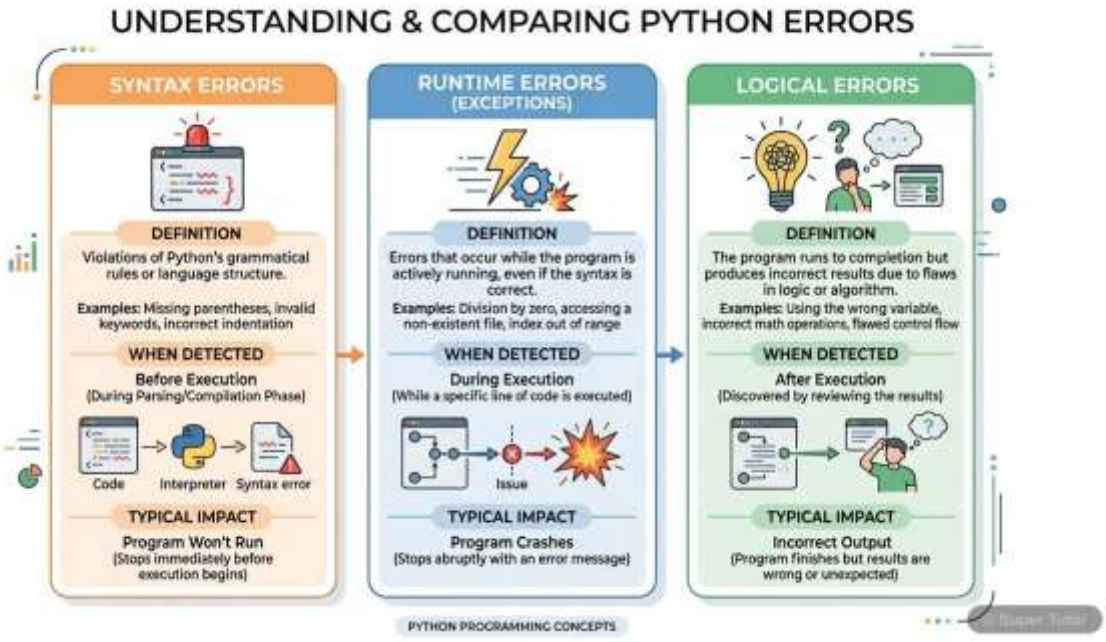
Actual:

-10

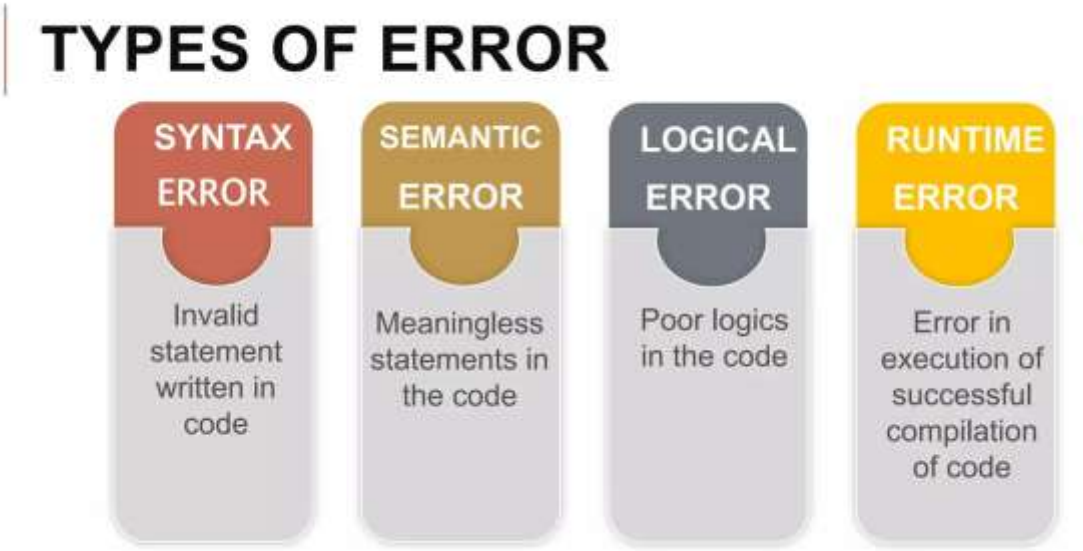
This is a **logical error**.

4. Types of Programming Errors

3DWEBSOFT FOUNDATION



Error Type	Description	Example
Syntax Error	Typing mistake	Missing colon : in Python
Runtime Error	Program crashes	Dividing by zero
Logical Error	Wrong output	Wrong formula or logic



The three important categories are:

PROGRAM ERRORS

|

| | |

Syntax Runtime Logical

Error Error Error

Error	When it happens	Example
Syntax Error	Python grammar is incorrect	Missing :
Runtime Error	Program crashes while running	Divide by zero
Logical Error	Program runs but gives wrong result	Wrong formula

5. Syntax Errors

A **syntax error** occurs when Python's rules are not followed.

3DWEBSOFT FOUND

Example

```
if age >= 18
   print("Adult")
```

The colon is missing.

Correct:

```
if age >= 18:
   print("Adult")
```

Another example

Incorrect:

```
print("Hello"
```

Correct:

```
print("Hello")
```

Common syntax mistakes

- Missing :
- Missing)
- Missing quotation marks
- Incorrect indentation
- Misspelled Python keywords
- Incorrect brackets

6. Runtime Errors

A runtime error occurs while the program is executing.

Example:

3DWEBSOFT FOUNDATION

number = 10

result = number / 0

Python produces:

ZeroDivisionError

The syntax is valid, but the operation is impossible.

Other examples

int("hello")

→ ValueError

numbers = [10, 20]

print(numbers[5])

→ IndexError

with open("missing.txt") as file:

    data = file.read()

→ FileNotFoundError if the file doesn't exist.

7. Logical Errors

Logical errors are often more difficult because the program **runs without producing an error message**.

Example:

marks = 80

if marks >= 90:

    grade = "A"

elif marks >= 70:

3DWEBSOFT FOUNDATION

```
    grade = "B"
else:
    grade = "C"
print(grade)
```

This program may be correct depending on the grading rules.

But suppose the intended rule was:

90+ → A

80+ → B

70+ → C

Then the logic needs to be changed.

Important point

A program can run successfully and still be wrong.

8. Understanding Python Tracebacks

When Python encounters an unhandled exception, it displays a **traceback**.

Example:

```
a = 10
b = 0
print(a / b)
```

Python may display something similar to:

```
Traceback (most recent call last):
  File "program.py", line 3, in <module>
```

3DWEBSOFT FOUNDATION

```
print(a / b)
```

ZeroDivisionError: division by zero

Students should learn to read it from the bottom.

Important information

File name

↓

Line number

↓

Problematic statement

↓

Exception type

↓

Error message

9. How to Read an Error Message

Consider:

ValueError: invalid literal for int() with base 10

The important part is:

ValueError

This tells us the **type of problem**.

The message tells us **what went wrong**.

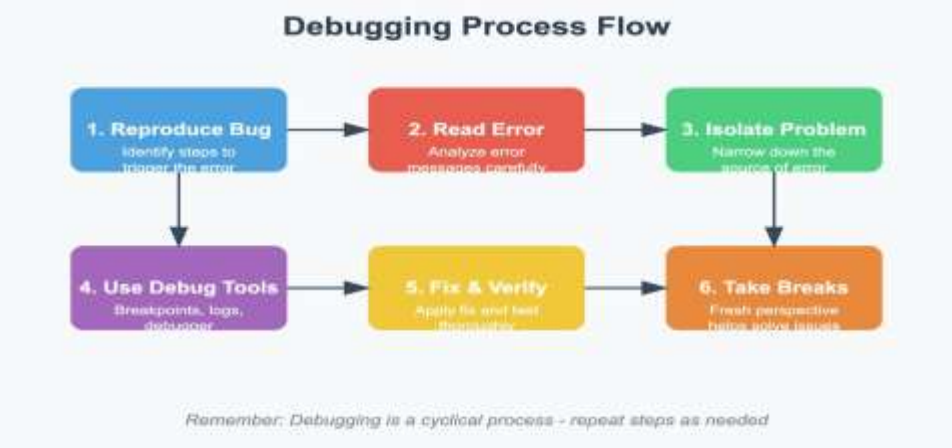
Debugging habit

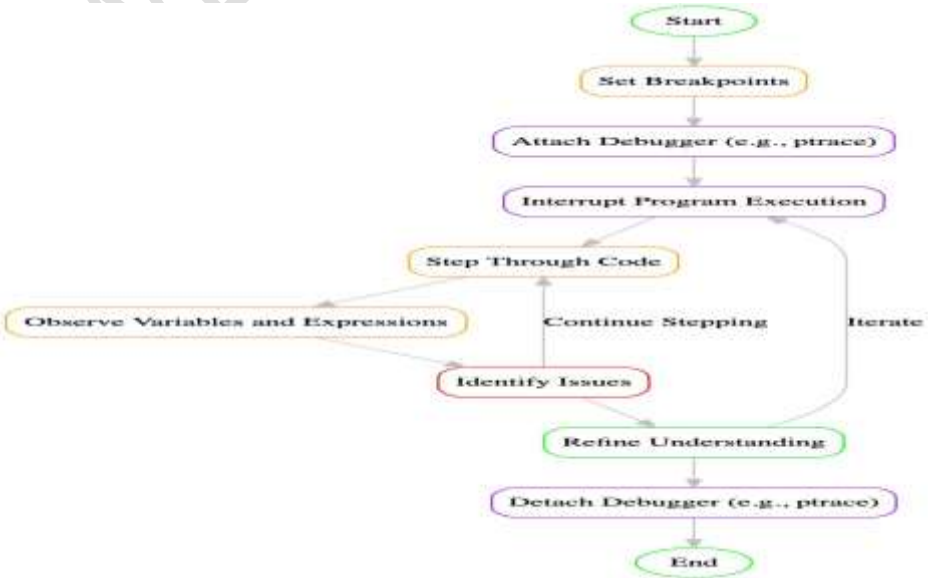
When you see an error:

3DWEBSOFT FOUNDATION

1. Don't panic.
2. Read the last line.
3. Identify the exception type.
4. Look at the line number.
5. Inspect that line.
6. Check nearby code.
7. Fix the cause.
8. Run the program again.

10. The Debugging Process





Use this seven-step process:

Step 1 – Reproduce the problem

Run the program and observe the error.

Step 2 – Read the error

Look at the traceback.

Step 3 – Locate the line

Find the line number mentioned by Python.

Step 4 – Identify the cause

Ask:

Why is this line failing?

Step 5 – Fix the code

Make the smallest appropriate correction.

Step 6 – Test again

Run the program.

Step 7 – Test different inputs

Make sure the fix didn't create another problem.

SDWEBSOFT FOUNDATION

11. Common Error 1 – SyntaxError

Buggy code

```
name = input("Enter your name: "
print(name)
```

Problem

Closing) is missing.

Correct code

```
name = input("Enter your name: ")
print(name)
```

12. Common Error 2 – IndentationError

Python uses indentation to define blocks.

Incorrect

```
age = 18

if age >= 18:
print("Adult")
```

Correct

```
age = 18

if age >= 18:
    print("Adult")
```

Important

Usually use **4 spaces** for one indentation level.

13. Common Error 3 – NameError

A NameError occurs when Python cannot find a variable or name.

Incorrect

```
name = "Rahul"
```

SUWEBSOFT FOUNDATION

print(nam)

The variable is called name, not nam.

Correct

name = "Rahul"

print(name)

Debugging Tip

Check:

- Spelling
- Capitalization
- Whether the variable was created
- Variable scope

14. Common Error 4 – TypeError

Example:

age = 17

print("Age: " + age)

This causes a TypeError because Python cannot directly combine a string and an integer using +.

Fix 1

print("Age:", age)

Fix 2

3DWEBSOFT FOUNDATION

```
print("Age: " + str(age))
```

15. Common Error 5 – ValueError

Example:

```
age = int("abc")
```

Python cannot convert "abc" into an integer.

Safe version

```
try:
    age = int(input("Enter age: "))
    print("Age:", age)

except ValueError:
    print("Please enter a valid number.")
```

This connects directly with **Class 17 – Exception Handling**.

16. Common Error 6 – ZeroDivisionError

Buggy code

```
a = 100
b = 0

print(a / b)
```

Fix using validation

```
a = 100
```

3DWEBSOFT FOUNDATION

```
b = 0
if b != 0:
    print(a / b)
else:
    print("Cannot divide by zero.")
```

Or use exception handling:

```
try:
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

17. Common Error 7 – IndexError

Buggy code

```
students = ["Rahul", "Priya", "Arjun"]


print(students[5])
```

Valid indexes are:

0 → Rahul

1 → Priya

2 → Arjun

There is no index 5.

Fix

```
students = ["Rahul", "Priya", "Arjun"]
```

3DWEBSOFT FOUNDATION

```
print(students[2])
```

Or safely check:

```
index = 5

if 0 <= index < len(students):
    print(students[index])
else:
    print("Invalid index.")
```

18. Common Error 8 – KeyError

Buggy code

```
student = {
    "name": "Rahul",
    "marks": 85
}

print(student["age"])
```

There is no "age" key.

Fix

```
print(student.get("age", "Age not available"))
```

This is a useful debugging and defensive-programming technique.

19. Common Error 9 – FileNotFoundError

3DWEBSOFT FOUNDATION

Buggy code

```
with open("students.txt", "r") as file:
    print(file.read())
```

If the file does not exist, Python raises:

FileNotFoundError

Fix

Check whether the file exists or handle the exception:

```
try:
    with open("students.txt", "r") as file:
        print(file.read())

except FileNotFoundError:
    print("students.txt was not found.")
```

20. Common Error 10 – JSONDecodeError

This is particularly useful because students learned JSON in **Class 16**.

Suppose students.json contains invalid JSON.

```
import json

try:
    with open("students.json", "r") as file:
        students = json.load(file)
```

3JWEBSOFT FOUNDATION

```
except json.JSONDecodeError:
    print("Invalid JSON data.")
```

Debugging Tip

Check:

- Missing commas
- Missing quotation marks
- Incorrect brackets
- Invalid JSON values
- Extra characters

21. Debugging with print()

One of the simplest debugging techniques is to use print().

Example:

```
marks = 80
bonus = 10

print("marks =", marks)
print("bonus =", bonus)

total = marks + bonus

print("total =", total)
```

Output:

SUWEBSOFT FOUNDATION

marks = 80

bonus = 10

total = 90

This helps us understand what values the program is using.

22. Debugging with Checkpoints

Students can place temporary messages inside the program.

```
print("Step 1")
name = input("Enter name: ")
print("Step 2")
marks = int(input("Enter marks: "))
print("Step 3")
print(name, marks)
```

If the program stops after:

Step 2

we know the problem occurred while reading/converting the marks.

Debugging idea

Step 1 → Working

Step 2 → Working

Step 3 → Not reached

Problem is between Step 2 and Step 3.

23. Debugging a Loop

3DWEBSOFT FOUNDATION

Consider:

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

If a student expected 1 to 5, the program isn't necessarily broken—the programmer's understanding of range() may be wrong.

Correct:

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Debugging technique

Print the loop variable:

```
for i in range(5):
```

3DWEBSOFT FOUNDATION

```
print("i =", i)
```

24. Debugging Functions

Buggy program

```
def calculate_total(price, quantity):
    return price * quantity


result = calculate_total(100)
print(result)
```

The function requires two arguments, but only one is provided.

Python reports a TypeError.

Correct

```
def calculate_total(price, quantity):
    return price * quantity
result = calculate_total(100, 3)
print(result)
```

Output:

300

Debugging question

Always check:

What arguments does my function expect?

25. Debugging with Assertions

3DWEBSOFT FOUNDATION

Python provides:

assert

An assertion checks whether something is true.

Example:

marks = 85

assert 0 <= marks <= 100

print("Valid marks")

If the condition is false:

marks = 150

assert 0 <= marks <= 100

print("Valid marks")

Python raises an AssertionError.

Use

Assertions are useful for catching assumptions during development.

26. Using a Debugger

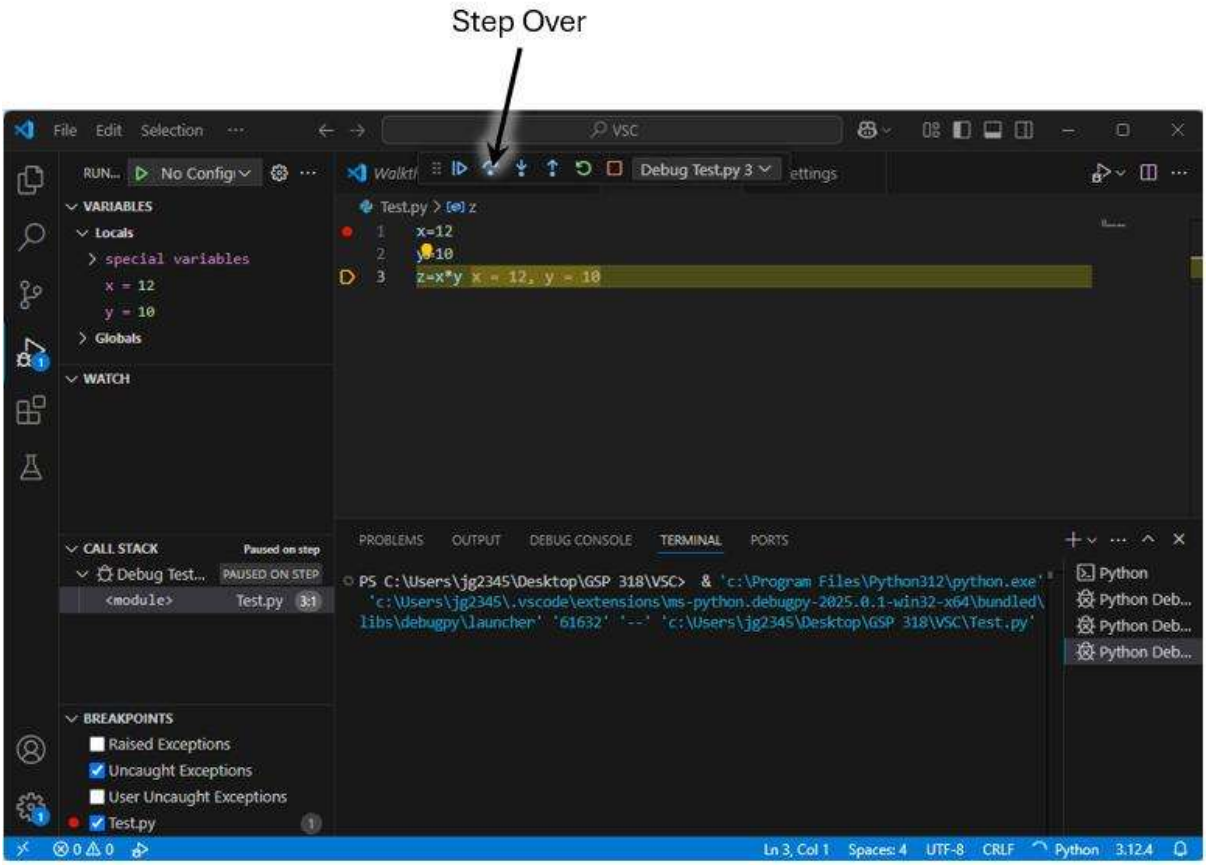
Modern Python development environments provide debugging tools.

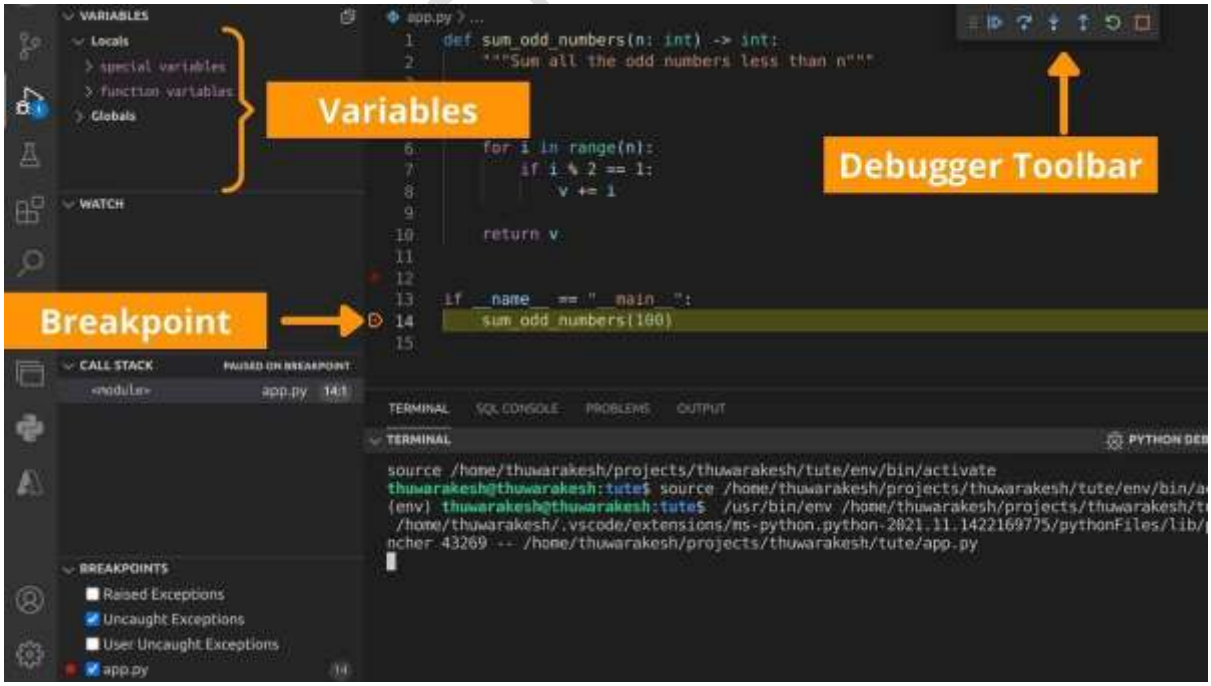
A debugger allows programmers to:

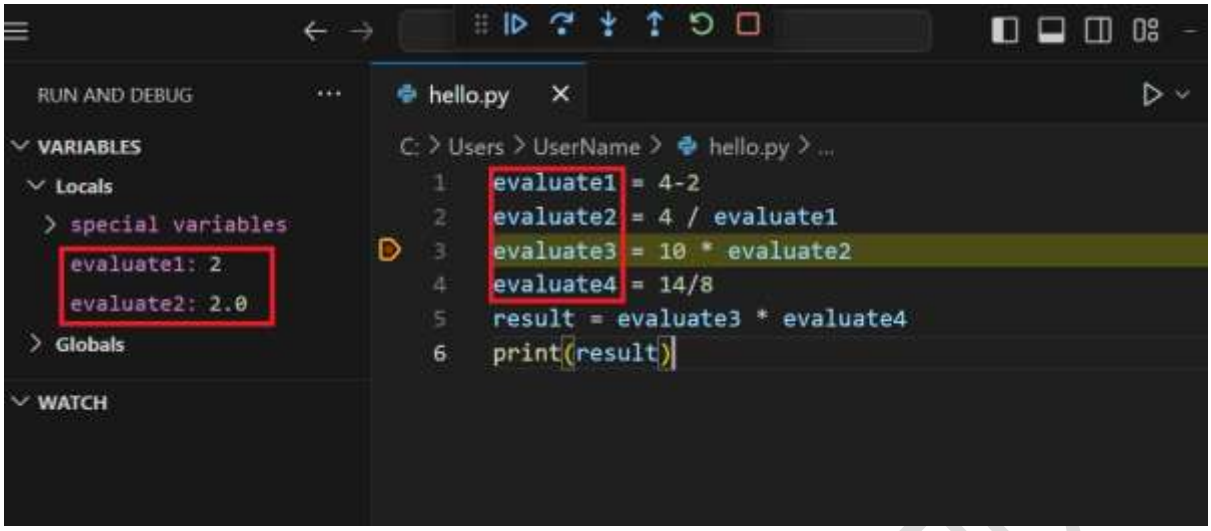
- Pause a program.
- Inspect variables.
- Execute code step by step.
- Find where values change.
- Set breakpoints.

SUWEBSOFT FOUNDATION

- Understand program flow.







27. What is a Breakpoint?

A **breakpoint** tells the debugger:

"Pause the program here."

Example:

name = "Rahul"

marks = 85

total = marks + 10

print(total)

A breakpoint can be placed on:

total = marks + 10

When the program reaches that line, execution pauses.

Students can inspect:

name → "Rahul"

marks → 85

and then continue execution.

3DWEBSOFT FOUNDATION

28. Step-by-Step Debugging

A debugger usually provides controls such as:

Command	Meaning
Continue	Continue execution
Step Over	Execute current line
Step Into	Enter a function
Step Out	Leave current function
Restart	Run again
Stop	End debugging

Simple idea

Line 1 → Execute

Line 2 → Execute

Line 3 → PAUSE

Line 4 → Execute

This makes complex programs easier to understand.

29. Practical Debugging Example

Consider this buggy program:

```
marks = [80, 90, 70]

total = 0

for i in range(1, 4):

    total += marks[i]
```

3DWEBSOFT FOUNDATION

```
average = total / 3
print("Average:", average)
```

Problem

List indexes are:

0

1

2

But the loop attempts:

1

2

3

marks[3] causes IndexError.

Correct program

```
marks = [80, 90, 70]
total = 0
for i in range(3):
    total += marks[i]
average = total / 3
print("Average:", average)
```

Output:

Average: 80.0

30. Debugging a Student Marks Program

3JWEBSOFT FOUNDATION

Buggy version

```
name = input("Enter name: ")
marks = int(input("Enter marks: "))
if marks > 40:
    result = "Fail"
else:
    result = "Pass"
print(name, result)
```

Find the logical error

The condition is reversed.

If marks are greater than 40, the student should normally pass.

Correct version

```
name = input("Enter name: ")
marks = int(input("Enter marks: "))
if marks >= 40:
    result = "Pass"
else:
    result = "Fail"
print(name, result)
```

Example

```
Enter name: Rahul
Enter marks: 75
Rahul Pass
```

3DWEBSOFT FOUNDATION

31. Debugging Strategy: Divide and Check

For a large program, don't inspect everything at once.

Break the program into sections:

Input

↓

Validation

↓

Processing

↓

Calculation

↓

Output

Check each section separately.

For example:

```
print("Input section working")
name = input("Name: ")
marks = int(input("Marks: "))
print("Processing section working")
percentage = marks
print("Output section working")
print(name, percentage)
```

This helps locate the problem.

SOWEBSOFT FOUNDATION

32. Common Debugging Checklist

When a program doesn't work, ask:

Syntax

- Did I forget :?
- Did I close brackets?
- Did I close quotation marks?
- Is indentation correct?

Variables

- Is the variable defined?
- Is the spelling correct?
- Did I use the correct variable?

Data

- Is the data type correct?
- Should I use int() or float()?
- Is the input valid?

Lists

- Is the index within range?

Dictionaries

- Does the key exist?

Files

- Does the file exist?
- Is the file path correct?

Logic

SUWEBSOFT FOUNDATION

- Is the condition correct?
- Is the formula correct?
- Is the loop range correct?

Functions

- Did I pass the correct arguments?
- Did the function return the expected value?

33. Debugging vs Exception Handling

Students often confuse these two concepts.

Debugging	Exception Handling
Finds and fixes bugs	Handles problems during execution
Mainly a development process	Part of program design
Uses debugging tools, print statements, tests	Uses try, except, else, finally
Helps locate the cause	Helps application respond safely

Example

Debugging:

Why am I getting ZeroDivisionError?

Exception handling:

What should my application do if division by zero occurs?

3DWEBSOFT FOUNDATION

Practical Session

34. Practical Activity – Find the Bugs

Give students this program:

```
students = ["Rahul", "Priya", "Arjun"]


for i in range(1, 4):
    print(students[i])
```

Ask students:

1. What error occurs?
2. Which line causes it?
3. Why?
4. How can we fix it?

Correct answer

```
for i in range(3):
    print(students[i])
```

Or better:

```
for student in students:
    print(student)
```

35. Practical Activity – Debug the Calculator

Buggy code

```
a = int(input("Enter first number: "))
```

3DWEBSOFT FOUNDATION

```
b = int(input("Enter second number: "))
choice = input("Enter + or -: ")
if choice == "+":
    result = a - b
elif choice == "-":
    result = a + b
print("Result:", result)
```

Challenge

Find the logical errors.

Correct code

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
choice = input("Enter + or -: ")
if choice == "+":
    result = a + b
elif choice == "-":
    result = a - b
else:
    print("Invalid operation.")
    result = None
print("Result:", result)
```

36. Mini Project – Debugging Challenge

3DWEBSOFT FOUNDATION

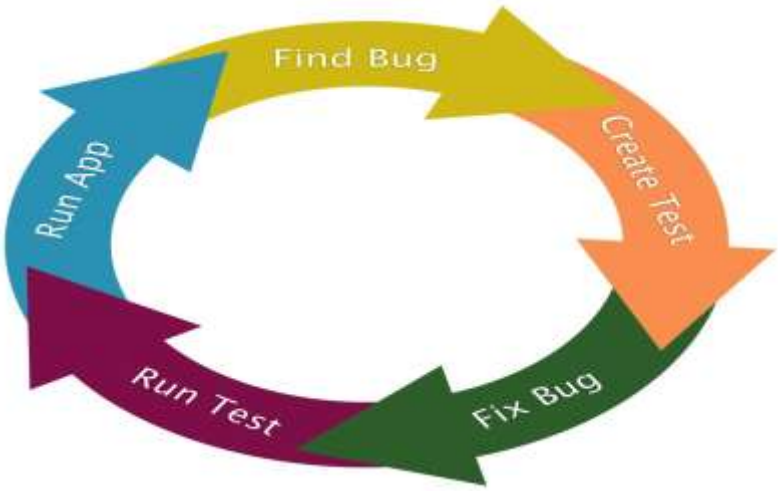
Project: Error-Free Student Application

Students should take the **JSON Student Management Application from Classes 16–17** and intentionally test it with different inputs.

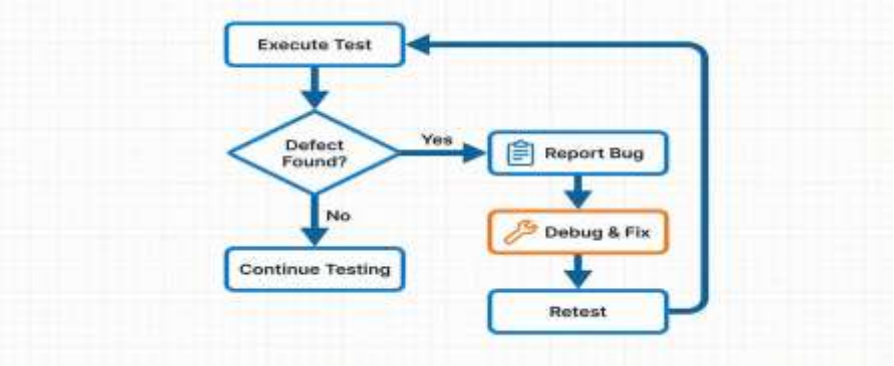
Test cases

	Test Input	Expected behavior
1	Valid roll number	Student found
2	Text instead of roll number	Friendly error
3	Missing JSON file	Create/use empty database
4	Invalid JSON	Display JSON error
5	Marks = 150	Reject marks
6	Negative marks	Reject marks
7	Unknown student	Display not found
8	Duplicate roll number	Display validation message

Project Flow



3D WEBSOFT FOUNDATION



Student Application

↓

Test

↓

Does it work?

/ \

YES NO

| |

↓ ↓

Finish Find Bug

↓

Understand

↓

Fix

↓

Test

↓

3DWEBSOFT FOUNDATION

Repeat

37. Debugging Exercise Set

Exercise 1

Find the error:

name = "Rahul"

print(nam)

Error: NameError

Exercise 2

numbers = [10, 20, 30]

print(numbers[4])

Error: IndexError

Exercise 3

a = 10

b = 0

print(a / b)n

Error: ZeroDivisionError

Exercise 4

age = int("abc")

Error: ValueError

3DWEBSOFT FOUNDATION

Exercise 5

```
if 10 > 5
   print("Correct")
```

Error: SyntaxError

Exercise 6

```
student = {"name": "Rahul"}
print(student["marks"])
```

Error: KeyError

38. Classroom Group Activity

Divide students into small teams.

Give each team a program containing **3–5 bugs**.

Students must create a table:

Bug	Type	Cause	Solution
Missing :	Syntax	Python grammar	Add :
Wrong index	Runtime	Index outside list	Correct index
Wrong formula	Logical	Incorrect calculation	Fix formula

Team Challenge

The first team to correctly identify and explain all bugs wins.

39. Best Debugging Practices

3DWEBSOFT FOUNDATION

Practice 1 – Read the error carefully

Don't immediately change random lines.

Practice 2 – Reproduce the problem

Make sure you can see the problem consistently.

Practice 3 – Change one thing at a time

This makes it easier to identify what fixed the problem.

Practice 4 – Use meaningful variable names

Instead of:

x = 85

use:

student_marks = 85

Practice 5 – Test edge cases

For marks:

0

39

40

100

101

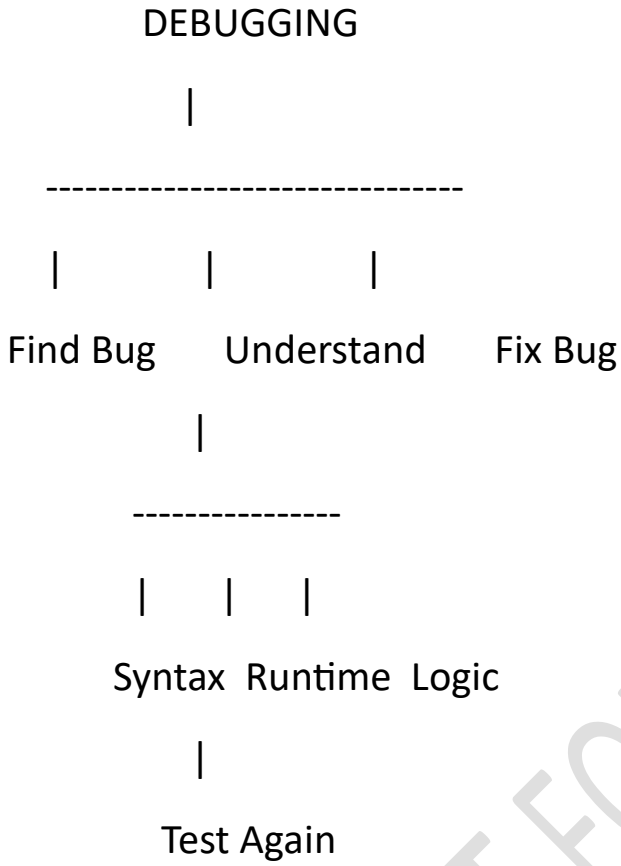
-1

Practice 6 – Use functions

Breaking large programs into smaller functions makes debugging easier.

3DWEBSOFT FOUNDATION

40. Quick Revision



Remember:

Don't just fix the error message. Find the reason behind the error.

41. Important Terms

Term	Meaning
Bug	Problem in a program
Debugging	Finding and fixing bugs
Syntax Error	Invalid Python syntax
Runtime Error	Problem during execution
Logical Error	Incorrect program logic

3DWEBSOFT FOUNDATION

Term	Meaning
Traceback	Information about an exception
Breakpoint	Point where debugger pauses
Debugger	Tool for inspecting program execution
Testing	Checking whether the program works correctly

42. Knowledge Check – MCQs

1. What is debugging?

A. Writing comments
B. Finding and fixing program problems
C. Installing Python
D. Creating variables

Answer: B

2. Which error occurs when Python syntax is invalid?

A. ValueError
B. SyntaxError
C. KeyError
D. IndexError

Answer: B

3. What type of error gives incorrect output but may not crash the program?

A. Syntax error
B. Logical error
C. File error
D. Import error

3DWEBSOFT FOUNDATION

Answer: B

4. Which error occurs when dividing by zero?

A. TypeError
B. ValueError
C. ZeroDivisionError
D. NameError

Answer: C

5. What does a breakpoint do?

A. Deletes a program
B. Stops execution at a selected point
C. Removes variables
D. Converts Python to JSON

Answer: B

6. Which is useful for checking variable values during debugging?

A. print()
B. input() only
C. import
D. return only

Answer: A

7. What should you usually inspect first when Python displays a traceback?

A. Computer wallpaper
B. Last line and exception type
C. Internet connection
D. Keyboard

Answer: B

3DWEBSOFT FOUNDATION

43. Practical Assignment

Build and Debug a Student Marks Program

Create a program that:

1. Accepts student name.
2. Accepts marks in five subjects.
3. Calculates total.
4. Calculates average.
5. Determines pass/fail.
6. Displays the result.

Then intentionally test:

Invalid marks

Negative marks

Marks greater than 100

Text instead of numbers

Empty input

Students must:

- Identify the errors.
- Record the error type.
- Fix the program.
- Add exception handling.
- Test the corrected program.

44. Final Takeaway

3DWEBSOFT FOUNDATION

Students should remember the **Debugging Cycle**:

RUN

↓

FIND BUG

↓

READ ERROR

↓

IDENTIFY CAUSE

↓

FIX CODE

↓

TEST AGAIN

↓

TEST EDGE CASES

3DWEBSOFT FOUNDATION